UNIVERSIDADE DE SÃO PAULO

Gabriel Whanderson Silva de Castro

Paulo Sergio Moura da Costa

Sandro Kenvi Mayorga Cáceres

MAC0216 – Técnicas de Programação I

RELATÓRIO TÉCNICO – TAREFA 2

São Paulo

2026

Repositório do Git

https://github.com/PsSergio/tecprog_tarefa2

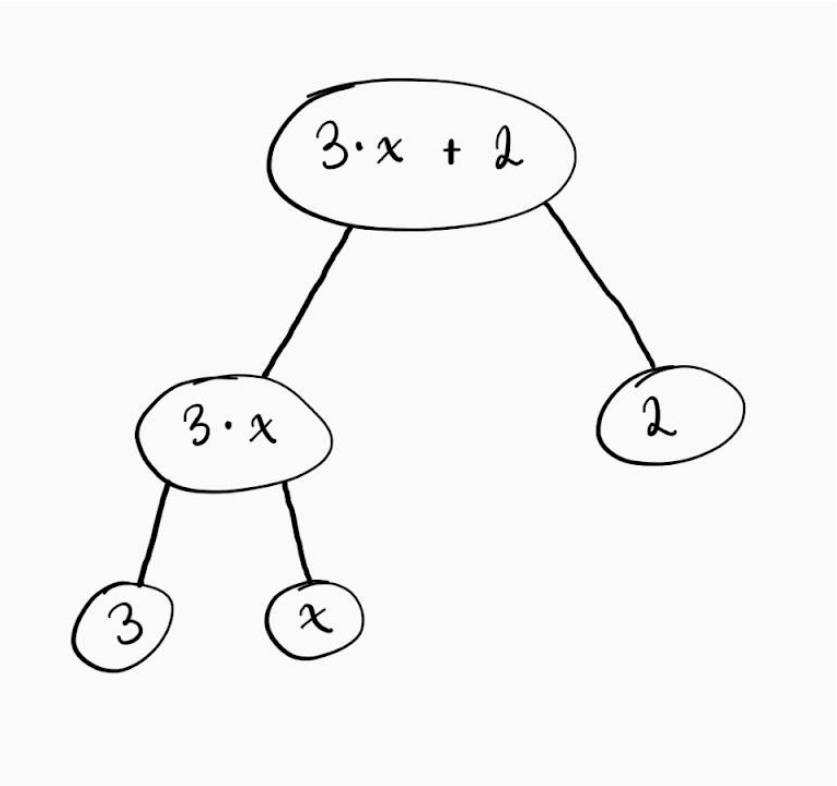
Domínio do Problema Resolvido pelo Projeto

O domínio escolhido foi Cálculo Diferencial I. Tal escolha foi motivada pelos estudos realizados no primeiro semestre, e a identificação da importância de analisar funções, máximos e mínimos. Nesse projeto, o objetivo central é fornecer ao usuário uma biblioteca que o ajude a analisar funções matemáticas de uma variável localmente.

Arquitetura do Software

A biblioteca se concentra em quatro módulos: **funcoes.h**, **derivadas.h**, **limites.h** e **representacao.h.** O primeiro é a base de como representamos expressões matemáticas, sendo de utilidade para manipulações destas. Por isso, vamos dar ênfase na explicação da estratégia para organizar as expressões.

O módulo funcoes.h mapeia funções matemáticas, recebidas em primeiro plano como uma string, em grafos. A estratégia consiste em particionar uma expressão recursivamente em no máximo mais dois vértices. Veja o exemplo a seguir:



Dessa maneira, a manipulação das equações consiste em manipular esses grafos. Cada vértice do grafo é alocado como uma struct Func, que possui propriedades que permitem identificar a própria expressão, relações com seus vértices seguintes e outras informações relevantes para a programação.

Organização e responsabilidade dos módulos

Como dito anteriormente, temos quatro módulos que abordam conceitos de Cálculo Diferencial I.

Módulo 1: funcoes

- **Responsabilidade:** Representação de expressões matemáticas em grafos, análise sintática de expressões, simplificação algébrica e avaliação numérica para pontos reais.
- **Funções Públicas:**

  1. **Func* mapear_func(const char *str):** Converte string em uma estrutura de grafos.
  2. **void liberar_func(Func *f):** Libera recursivamente toda a estrutura da heap.

3. **Func* simplificar_func(const Func *f):** Aplica regras de simplificação algébrica. (x-x = 0) por exemplo.
4. **double avaliar_func(const Func *f, double x):** Avalia numericamente a expressão em um ponto x dado; retorna NAN ou INFINITY em descontinuidades ou indefinições.

Módulo 2: derivadas

- **Responsabilidade:** Algoritmo recursivo de derivação por manipulações simbólicas sobre os grafos mapeados, aplicando regras do produto, quociente, potência e cadeia.
- **Funções Públicas:**
  5. **Func* derivar_func(Func *f, char *var):** Deriva a expressão de forma simbólica em relação à variável informada.
  6. **Func* reta_tangente_func(Func *f, double x0):** Constrói a expressão simbólica da reta tangente à curva no ponto x0 no formato y = mx + b.

Módulo 3: limites

- **Responsabilidade:** Resolução de limites analíticos e determinação de continuidade pontual.
- **Funções Públicas:**
  7. **Func* limite_func(Func *f, char var, double target, enum LimitSide side):** Calcula o limite bilateral ou unilateral (LEFT, RIGHT, BOTH). Trata indeterminações (0/0 e inf/inf) via aplicação recursiva de L'Hôpital combinada com derivar_func.
  8. **bool func_continua_em(Func *f, double x0):** Verifica a continuidade comparando o limite e o valor da função no ponto x0 com avaliar_func.

Módulo 4: representacao

- **Responsabilidade:** Visualização gráfica de funções de maneira simples (terminal) ou mais detalhada (com exportação).
- **Funções Públicas:**
  9. **void draw_graph(Func *f):** Desenha o plano cartesiano com eixos e marcadores de escala em modo texto diretamente no terminal.

Estratégia de versionamento com Git.

Organizamos nossos commits criando branchs para cada módulo. Temos então as branchs: feat/funcoes; feat/derivadas; feat/limites e feat/representacao. Assim, era criado um Pull Request para atualizar a main.

Dificuldades e Decisões do projeto

A principal decisão que fizemos foi representar as expressões matemáticas em grafos. Isso foi realizado no início do desenvolvimento, pois sabíamos que não poderíamos apenas manipular as strings de qualquer maneira. A decisão, em particular, foi considerada pelo grupo após o estudo de grafos em Álgebra Linear, e um acompanhamento adicional como ouvinte de aulas de algoritmos da Escola Politécnica.

No entanto, as maiores dificuldades foram manipular esses grafos. O processo de atualizar os vértices após uma modificação, por exemplo, foi um obstáculo, visto que estava presente em quase todo o projeto. Além disso, também tivemos bastante dificuldade, quando o projeto já estava consideravelmente grande em questão de código, de arrumar erros de liberação de memória corretamente. Isso aconteceu, pois feito uma modificação pequena, desencadeou uma série de erros pequenos que estavam “escondidos” anteriormente.

Estratégia e resultados dos testes

Separamos os testes em módulos, por exemplo, “test_derivadas.c”. Para a execução destes, criamos um arquivo test_runner.c que juntava a execução de todos os testes e acionado com o “make test”.

Exemplo de teste que inicialmente falhou e posterior correção

Um dos primeiros testes automatizados que fizemos foi com a função mapear_func. O teste apontou que a função não estava mapeando corretamente expressões matemáticas com espaço. Ou seja, “1 / x” estava dando erro, mas “1/x”, não. Então percebemos que realmente não foi levado em consideração essa possibilidade de entrada, e fizemos uma função “remover_espacos” para resolver esse problema.

Resultados obtidos e melhorias

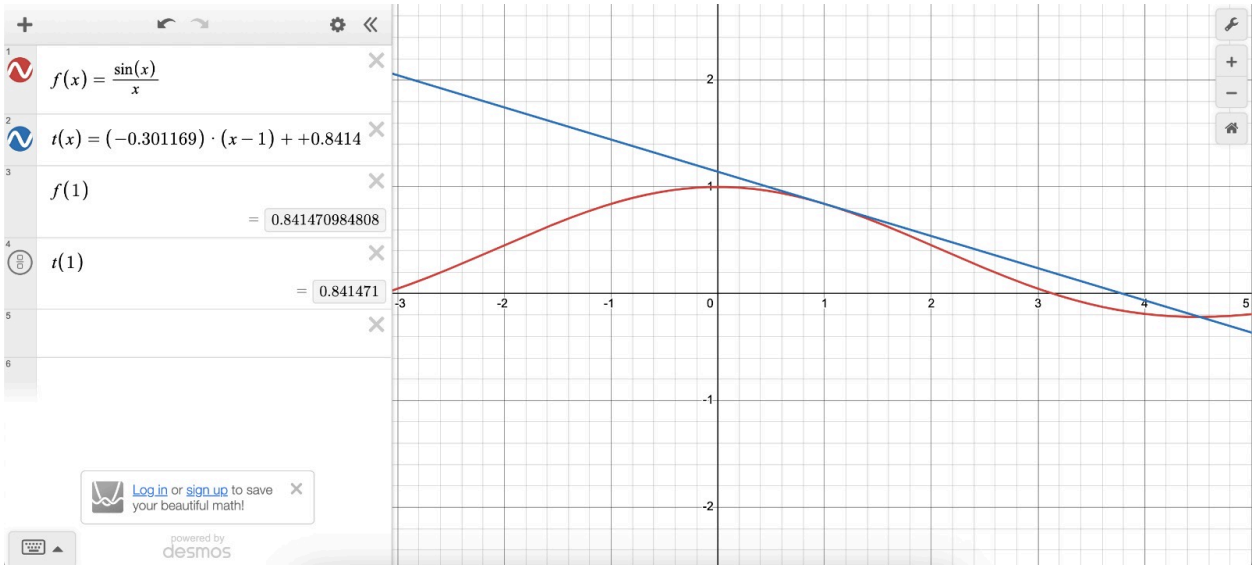
Conseguimos fazer grande parte do planejado. Apesar das dificuldades de manipulação de memória, por exemplo, os resultados estão precisos e dentro do esperado. Segue os resultados do nosso programa, e uma comparação com a página web Desmos, respectivamente:

```
ARQUIVO MAIN - BIBLIOTECA DE CÁLCULO 1

Módulo 1 - Funções
Função mapeada: f(x) = sen(x)/x
f(1) = 0.841471

Módulo 2 - Derivadas
f'(x) = ((cos(x))*(x)-sen(x))/((x)*(x))
Equação da reta tangente em (1.0): t(1) = (-0.301169)*(x-1.000000)+0.841471
Comparando f(1.0) com t(1.0): 0.841471 && 0.841471

Módulo 3 - Limites
sen(x)/x é contínua em 0.0 ? Resultado: Não
Lim (sen(x)/x) quando x-> 0.0 = 1.000000
```



A principal melhoria seria implementar a função ln(x). Para o atual projeto, configuramos as seguintes relações matemáticas: soma, subtração, multiplicação, divisão, sen(), cos(), tan(), potência. No entanto, a função ln(x) foi esquecida durante o desenvolvimento do projeto, e quando lembrada, não seria seguro implementá-la para demonstração do produto. Mas com essa melhoria, poderíamos calcular derivadas de a^x, por exemplo.